



Course Name Microcontroller Based System Design Lab

Course Code EEE 4706

Name	Student ID
------	------------

Munshaat Atiq Nuzaba	210021210
----------------------	-----------

Zahir Sadik	210021228
-------------	-----------

Abrar Mubasshir Rahman	210021230
------------------------	-----------

Faria Ahmed Suprova	210021236
---------------------	-----------

Fattaur Rahim	210021240
---------------	-----------

Musa Shuaibu	210021264
--------------	-----------

Group No 04

Project Name Bluetooth interfacing with Microcontroller

1. Objective

The objective of this project is to design and implement a multi-functional embedded system using the 8051 microcontroller (AT89S51/AT89C51). The system integrates Bluetooth communication, LCD feedback, relay control, and LED operations using 8051 Assembly language.

The key functionalities include:

- Wireless control of eight LEDs with ON/OFF and PWM brightness adjustment
- Morse code transmission via LED with real-time LCD display
- Wireless switching of two relay modules
- Caesar cipher decryption of Bluetooth-received text
- PIN-based password protection for secure access
- Implementation of interrupt-driven serial communication and hardware interfacing

2. Introduction

Embedded systems are fundamental to modern electronics, and the 8051 microcontroller remains widely used for learning due to its simple architecture and direct hardware control. Programming in Assembly provides detailed insight into low-level operations.

In this project, an HC-05 Bluetooth module is interfaced with the 8051 UART, allowing wireless command input from a smartphone. Commands are handled using an interrupt-driven circular buffer to ensure reliable communication.

The system operates in four modes:

- **Mode 1:** Relay control (P3.2, P3.3) for basic home automation
- **Mode 2:** Morse code transmission using LEDs with timing-based encoding and LCD output
- **Mode 3:** Caesar cipher decryption using a user-defined key
- **Mode 4:** LED control with PWM-based brightness adjustment

A PIN-based authentication system using a keypad (P2) restricts access, with a lockout after three failed attempts. A 16×2 LCD (P0, P3.5–P3.7) provides system feedback.

All firmware is written in Assembly, using internal RAM for variable storage and interrupt routines for UART communication. Hardware connections are mapped clearly across ports for efficient control.

3. Required Components

The following table lists all components used in this project along with their quantities and estimated costs:

#	Component	Specification	Quantity	Unit Price (BDT)	Total Price (BDT)
1	Microcontroller	AT89S52 (8051)	1	-	Collected
2	LCD Display	16x2 Character LCD	1	150	150
4	LED	5mm Red LED	10	5	50
6	Resistor	330 Ω (for LEDs)	10	2	20
7	Resistor	10k Ω (pull-up)	5	2	10
11	Push Button / Switch	Tactile Switch	2	5	10
13	Breadboard / PCB	Standard	3	50	150

#	Component	Specification	Quantity	Unit Price (BDT)	Total Price (BDT)
14	Jumper Wires	Male-to-Male	50	2	100
15	Power Supply	5V DC	1	100	100
Total					590

4. Circuit Diagram

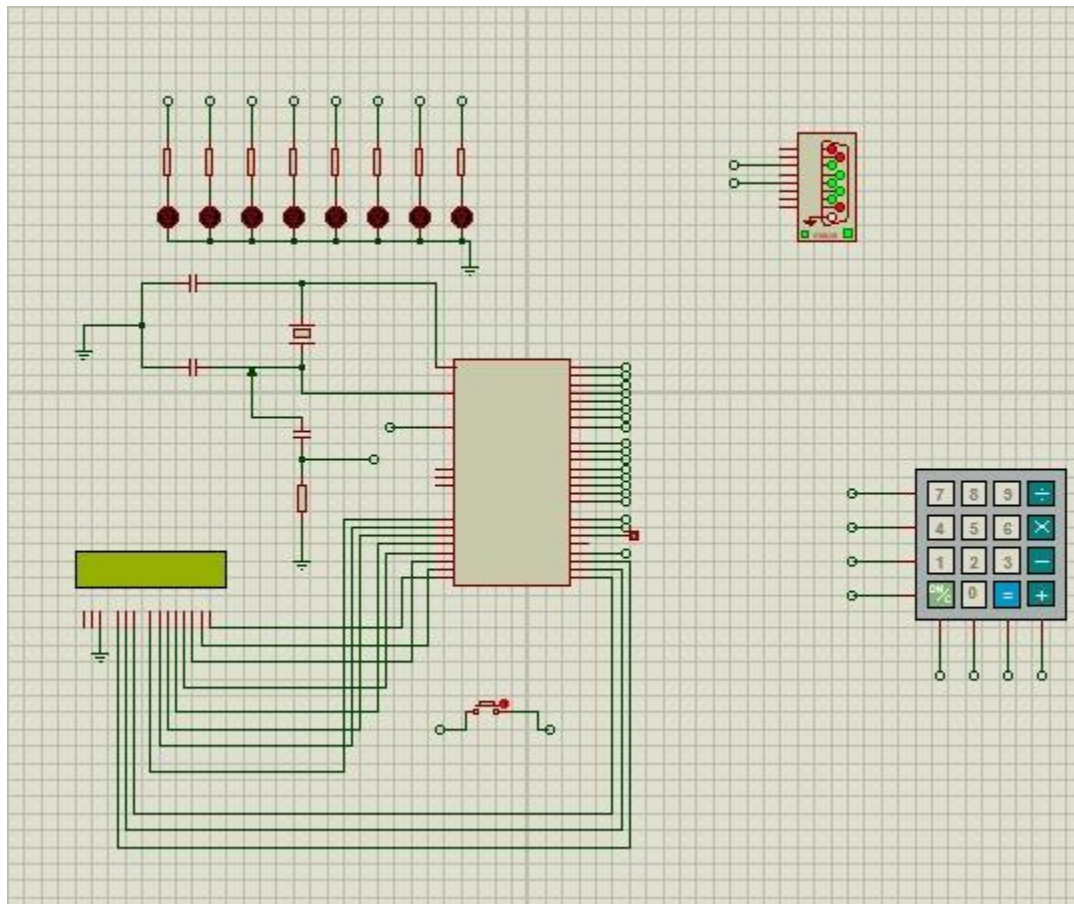


Figure 1: Proteus Circuit Simulation

The circuit is designed around the AT89S52 8051 microcontroller as the central processing unit. The key connections are as follows:

- **Microcontroller (AT89S52):** The main IC is driven by an 11.0592 MHz crystal oscillator connected to pins XTAL1 and XTAL2, with two 22pF capacitors for stabilization. The RST pin is connected through a 10k Ω resistor and 10 μ F capacitor for power-on reset.
- **16x2 LCD Display:** Connected to Port 2 (P2.0–P2.7) of the microcontroller for 8-bit data communication. The RS, RW, and EN control pins are connected to Port 3. A 10k Ω potentiometer is connected to the VEE pin (Pin 3) to adjust display contrast.
- **4x4 Matrix Keypad:** The rows and columns of the keypad are interfaced with Port 1 (P1.0–P1.7). The microcontroller scans the keypad to detect key presses, allowing the user to select operating modes (1R, 2M, 3E, 4LED as shown on the LCD).
- **LED Array (7 LEDs):** Seven LEDs are connected to the upper lines of Port 0 through 330 Ω current-limiting resistors to protect them from excessive current. The LEDs are used as output indicators.
- **7-Segment Display:** Connected to the microcontroller for displaying numeric output, driven through the available port pins.
- **Push Button / Switch:** A tactile switch is connected to one of the interrupt/input pins of the microcontroller, used to trigger specific operations or reset states.

Power Supply: The entire circuit operates on a regulated 5V DC supply. The VCC and GND rails are distributed to all components accordingly.

5. Mandatory features:

8 LED Control via Bluetooth:

Using a Bluetooth-connected device (such as a smartphone), the user can send commands to control all 8 LEDs connected to the microcontroller. Each LED can be turned on or off individually or all at once, demonstrating basic wireless GPIO control over a serial Bluetooth communication link.

Morse Code Transmission:

The user selects one of the 8 LEDs to act as the Morse code transmitter and sends a message (such as a name) from the Bluetooth device. The chosen LED blinks in the correct Morse code timing pattern — short blinks for dots and long blinks for dashes. Simultaneously, the LCD updates in real time, showing "dot" or "dash" on the first line as each symbol is transmitted, and displaying the full coded character sequence on the second line.

2-Relay Light Control via Bluetooth:

Two relay modules are interfaced with the microcontroller and connected to external lights or appliances. Through Bluetooth commands, the user can independently switch each relay on or off, simulating a simple home automation or remote switching scenario. This demonstrates the microcontroller's ability to drive higher-voltage loads safely through relay switching.

Encrypted Message Display:

The user sends an encrypted text string from their Bluetooth device to the microcontroller. The microcontroller applies a decryption algorithm (such as a Caesar cipher, where each letter is shifted by a fixed number) to recover the original plaintext, which is then displayed on the LCD screen. For example, sending "MJWFSQPPM" results in "LIVERPOOL" being shown on the display.

Additional Features:

Password Protection:

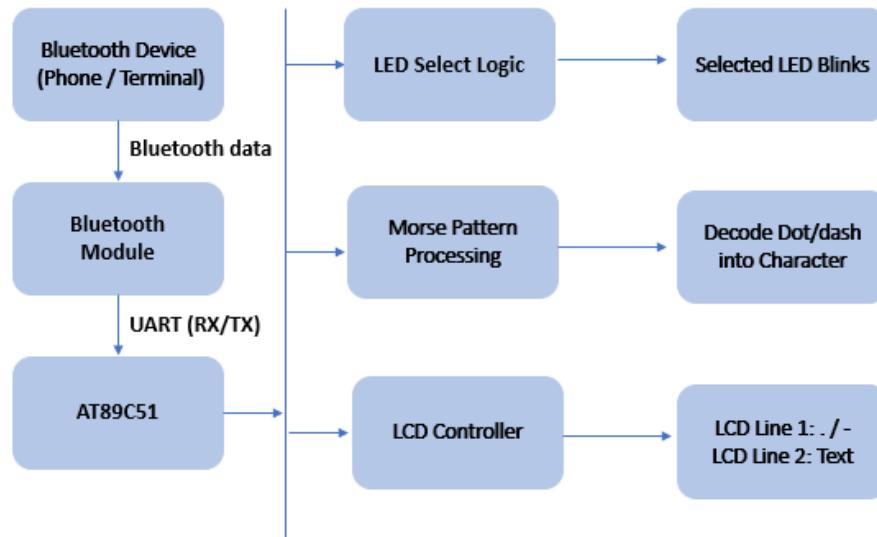
To prevent unauthorized access to the system, a password protection mechanism is implemented. When the system starts up or when a protected command is issued, the user must first send the correct password via Bluetooth. Only upon successful authentication does the system allow further control commands to be executed, adding a basic but practical layer of security to the project.

LED Brightness Control:

Beyond simple on/off control, the brightness of the LEDs can be adjusted dynamically using Pulse Width Modulation (PWM). The user sends a brightness level command from the Bluetooth device, and the microcontroller adjusts the PWM duty cycle accordingly, smoothly varying the light intensity from dim to full brightness. This feature demonstrates analog-style output control through a digital interface.

6. Working Principle:

Bluetooth control of 8 LEDs and Morse code transmission

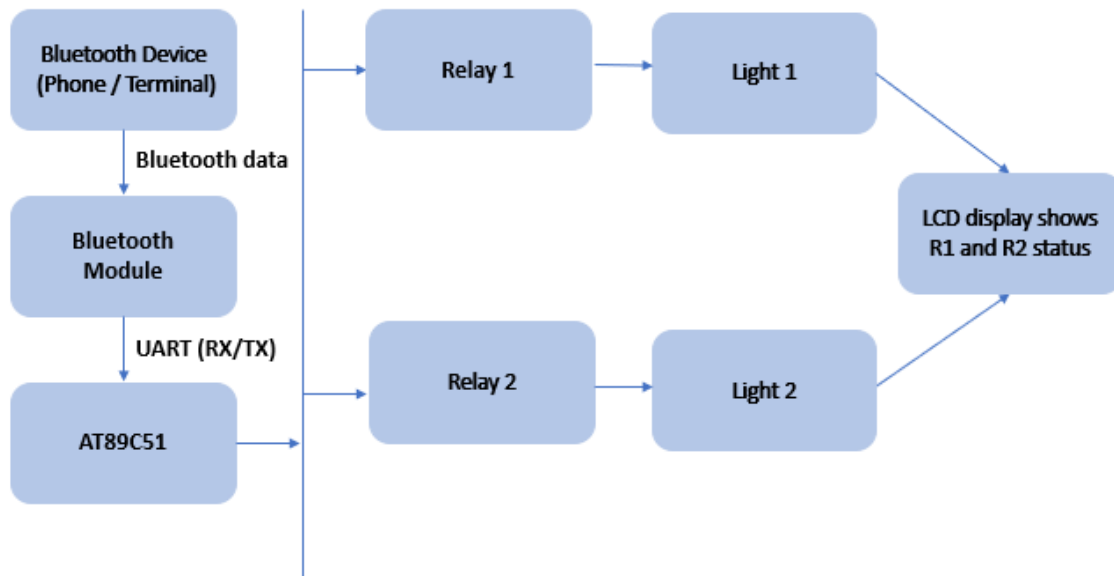


In this part, Bluetooth device controls 8 LEDs and is also used to send Morse code using one LED. First a number is sent from 1 to 8 by using the Bluetooth module, which selects the LED to be used to send the Morse code. After that symbols are sent

- . means dot
- - means dash
- Space means end of a letter
- / means space between words

When . is sent LED turns ON for a short time and when – is sent LED turns ON for a long time. Here the LED is sending the Morse code visually. The LCD display shows two things, on the first line it shows the current symbol and on the second line it shows the decoded message. The system stores the sequence of dots and dashes and when a space is pressed, the sequence is converted into letter. Then it displays the letter on the LCD.

Controlling 2 lights with 2 relays using Bluetooth

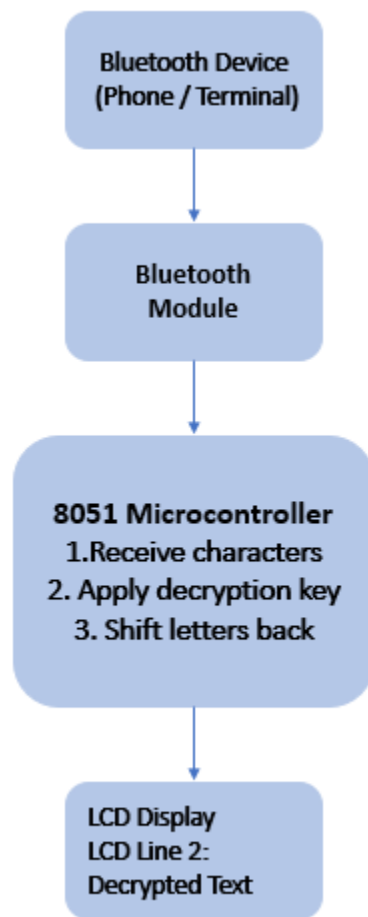


Here the Bluetooth device sends commands to the microcontroller and it reads those commands, then the microcontroller turns Relay 1 or Relay 2 ON or OFF. Each relay is connected to a light so when a relay turns ON, the corresponding light turns ON. Here

- Relay 1 → P3.2
- Relay 2 → P3.3

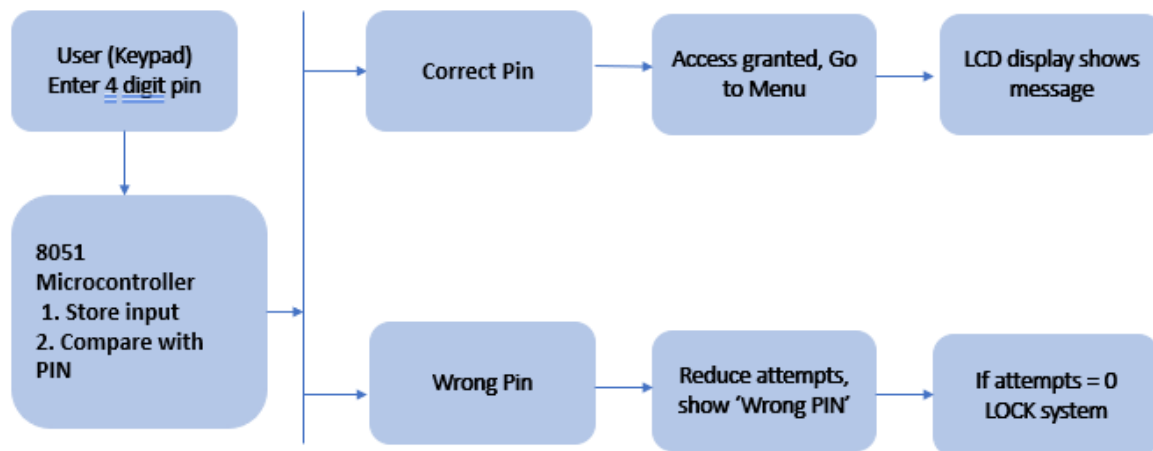
Relay works like an electrical switch. As the microcontroller can not directly power a light, so it controls the relay coil. LCD shows the relay status.

Working Principle (Encryption / Decryption using Bluetooth)



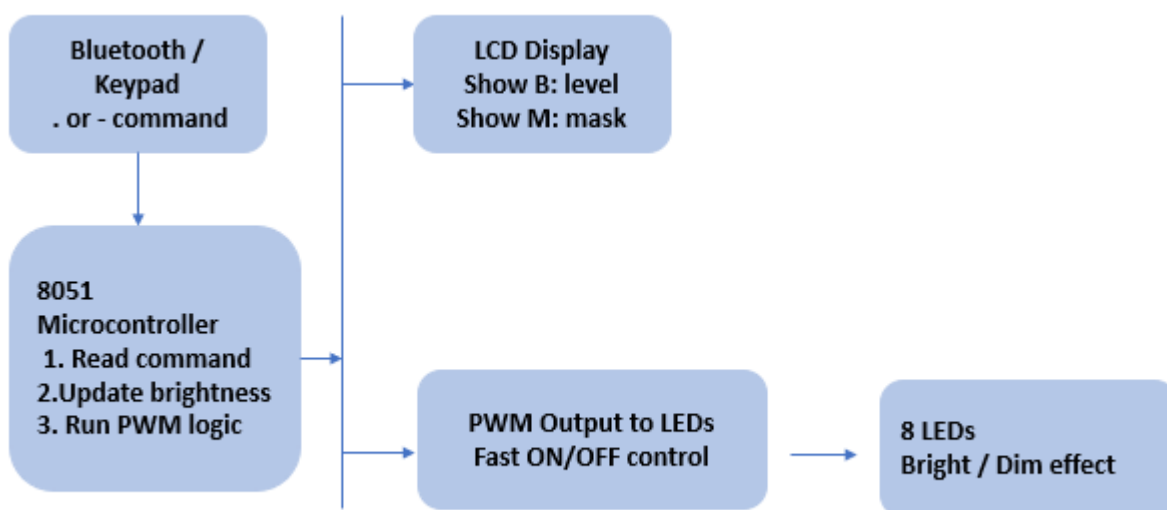
The user will first enter the decryption key and send an encrypted word through the Bluetooth module. Microcontroller will receive each character through serial communication and it reads the message one character at a time. The decryption process uses a simple shifting method, it shifts each letter backward by the key value. Each decrypted letter is shown on LCD display line 2.

Working Principle (Password Protection)



When the system is turned ON, LCD shows a welcome message and asks for pin by “Enter PIN”. When the user enters a 4 digit password using the keypad, each key pressed is stored in memory and shown as * on LCD for security. Then the microcontroller checks the password by comparing the entered digits with stored password. If password is correct then LCD will show Access Granted message and will go to menu. In case of wrong password, it will show Wrong PIN and the user has 3 attempts for correct input. If all attempts are used, system becomes locked.

Working Principle (LED Brightness Control)



This feature allows the user to control LED brightness using PWM. When the user selects mode 4, the system clears all LEDs, sets the LED mask to zero, sets an initial brightness value and shows

the LED control screen on the LCD. Microcontroller uses PWM method to control brightness which means LED is turned ON and OFF very quickly. If it stays ON for more time, it looks brighter and if it stays ON for less time, it looks dimmer. In that way the eye sees different brightness levels even though the LED is switching ON and OFF fast. When the user sends ., the program checks whether brightness is already at maximum. If not, it increases led4_bright by 1 and LCD value is updated with the new value. When the user sends – the program checks whether brightness is zero, if not it decreases led4_bright by 1 and LCD is updated again.

7. Code

```
; =====  
  
; EEE 4706 Project - Clean Hardware Version  
  
; Hardware map taken from user's known-good hardware code  
  
; -----  
  
; LCD data    : P0  
  
; LCD ctrl    : RS=P3.5, RW=P3.6, E=P3.7  
  
; Keypad      : P2  
  
; UART/HC-05  : RXD=P3.0, TXD=P3.1  
  
; Relay 1     : P3.2  
  
; Relay 2     : P3.3  
  
; LEDs 1..8   : P1.0..P1.7  
  
; -----  
  
; Modes:  
  
;   1 = Relay control  
  
;   2 = Morse + LED control  
  
;   3 = Encryption / Decryption  
  
;   4 = Advanced 8-LED + brightness control  
  
; =====  
  
$NOMOD51  
  
$INCLUDE (8051.MCU)
```

```

; -----

; PIN

; -----

PIN_D1      EQU      '1'

PIN_D2      EQU      '2'

PIN_D3      EQU      '3'

PIN_D4      EQU      '4'


; -----

; Hardware aliases

; -----

LCD_DATA    EQU      P0

RS          EQU      P3.5

RW          EQU      P3.6

E           EQU      P3.7

REL1        EQU      P3.2

REL2        EQU      P3.3


; -----

; RAM variables

; -----

pattern     EQU      30H

length      EQU      31H

m_led_mask  EQU      32H

m_led_num   EQU      33H

line2_pos   EQU      34H

enc_key     EQU      35H

```

tmp1	EQU	36H
tmp2	EQU	37H
rx_head	EQU	38H
rx_tail	EQU	39H
rx_count	EQU	3AH
led4_mask	EQU	3BH
led4_bright	EQU	3CH
led4_pwm	EQU	3DH

ORG 0000H

LJMP START

ORG 0023H

ISR_SERIAL:

PUSH ACC

PUSH PSW

PUSH 00H

JNB RI, ISR_DONE

MOV A, rx_count

CJNE A, #10H, ISR_STORE

MOV A, SBUF

CLR RI

SJMP ISR_DONE

ISR_STORE:

MOV R0, rx_head

```

        MOV     A, SBUF

        MOV     @R0, A

        CLR     RI

        INC     R0

        CJNE    R0, #60H, ISR_HEAD_OK

        MOV     R0, #50H

ISR_HEAD_OK:

        MOV     rx_head, R0

        INC     rx_count


ISR_DONE:

        POP     00H

        POP     PSW

        POP     ACC

        RETI


; =====

; START

; =====

START:

        MOV     SP, #5FH

        MOV     P0, #0FFH

        MOV     P1, #00H

        MOV     P2, #0FFH

        MOV     P3, #0FFH

        MOV     PSW, #00H

        LCALL   RXBUF_INIT

        LCALL   UART_INIT

```

```

        LCALL    LCD_INIT

        LJMP     SPLASH

; =====

; SPLASH / PASSWORD

; =====

SPLASH:

        LCALL    LCD_CLR

        MOV      A, #80H

        LCALL    COMNWRT

        MOV      DPTR, #MSG1

        LCALL    LCD_MSG

        MOV      A, #0C0H

        LCALL    COMNWRT

        MOV      DPTR, #MSG2

        LCALL    LCD_MSG

        LCALL    DELAY1S

        LCALL    DELAY1S

        LJMP     PASS_START


PASS_START:

        MOV      R6, #3


PASS_PROMPT:

        LCALL    LCD_CLR

        MOV      A, #80H

        LCALL    COMNWRT

        MOV      DPTR, #MSG_PIN

        LCALL    LCD_MSG

```

MOV A, #0C0H

LCALL COMNWRT

ACALL GET_KEY

MOV P2, #0FFH

MOV 40H, A

MOV A, #'*'

LCALL DATAWRT

MOV A, #' '

LCALL DATAWRT

ACALL GET_KEY

MOV P2, #0FFH

MOV 41H, A

MOV A, #'*'

LCALL DATAWRT

MOV A, #' '

LCALL DATAWRT

ACALL GET_KEY

MOV P2, #0FFH

MOV 42H, A

MOV A, #'*'

LCALL DATAWRT

MOV A, #' '

LCALL DATAWRT

ACALL GET_KEY


```

MOV     P2, #0FFH

MOV     43H, A

MOV     A, #'*'

LCALL   DATAWRT


LCALL   DELAY1S


MOV     A, 40H

CJNE    A, #PIN_D1, PASS_WRONG

MOV     A, 41H

CJNE    A, #PIN_D2, PASS_WRONG

MOV     A, 42H

CJNE    A, #PIN_D3, PASS_WRONG

MOV     A, 43H

CJNE    A, #PIN_D4, PASS_WRONG

```

PASS_OK:

```

LCALL   LCD_CLR

MOV     A, #80H

LCALL   COMNWRT

MOV     DPTR, #MSG_OK

LCALL   LCD_MSG

MOV     A, #0C0H

LCALL   COMNWRT

MOV     DPTR, #MSG_WELCOME

LCALL   LCD_MSG

LCALL   DELAY1S

LCALL   DELAY1S

```

LJMP MENU

PASS_WRONG:

DJNZ R6, PASS_LEFT

LCALL LCD_CLR

MOV A, #80H

LCALL COMNWRT

MOV DPTR, #MSG_LOCK

LCALL LCD_MSG

MOV A, #0C0H

LCALL COMNWRT

MOV DPTR, #MSG_WAIT

LCALL LCD_MSG

MOV R7, #10

PASS_WAIT_LOOP:

LCALL DELAY1S

DJNZ R7, PASS_WAIT_LOOP

LJMP PASS_START

PASS_LEFT:

LCALL LCD_CLR

MOV A, #80H

LCALL COMNWRT

MOV DPTR, #MSG_WRONG

LCALL LCD_MSG

MOV A, #0C0H

LCALL COMNWRT

MOV DPTR, #MSG_LEFT

```

        LCALL    LCD_MSG

        MOV      A, R6

        ADD      A, #'0'

        LCALL    DATAWRT

        LCALL    DELAY1S

        LJMP     PASS_PROMPT


; =====

; MENU

; =====

MENU:

        LCALL    LCD_CLR

        MOV      A, #80H

        LCALL    COMNWRT

        MOV      DPTR, #MSG_MENU1

        LCALL    LCD_MSG

        MOV      A, #0C0H

        LCALL    COMNWRT

        MOV      DPTR, #MSG_MENU2

        LCALL    LCD_MSG

        ACALL    GET_KEY

        MOV      P2, #0FFH

        CJNE     A, #'1', MENU_CHK2

        LJMP     RELAY_MODE

MENU_CHK2: CJNE     A, #'2', MENU_CHK3

        LJMP     MORSE_MODE

MENU_CHK3: CJNE     A, #'3', MENU_CHK4

        LJMP     ENCRYPT_MODE

```

```
MENU_CHK4:  CJNE    A, #'4', MENU
```

```
            LJMP     LEDCTRL_MODE
```

```
; =====
```

```
; MODE 1: RELAY CONTROL
```

```
; Keypad and Bluetooth both accepted
```

```
; 1=R1 ON 0=R1 OFF 3=R2 ON 2=R2 OFF 5=back
```

```
; =====
```

```
RELAY_MODE:
```

```
            LCALL    RXBUF_INIT
```

```
            CLR      REL1
```

```
            CLR      REL2
```

```
            LCALL    LCD_CLR
```

```
            MOV      A, #80H
```

```
            LCALL    COMNWRT
```

```
            MOV      DPTR, #MSG_RELAY1
```

```
            LCALL    LCD_MSG
```

```
            LCALL    RELAY_SHOW
```

```
            MOV      DPTR, #MSG_BT_REL_READY
```

```
            LCALL    UART_SEND_STR
```

```
RELAY_LOOP:
```

```
            LCALL    KEY_CHECK
```

```
            JNZ      RELAY_HAVE_CMD
```

```
            LCALL    BT_CHECK
```

```
            JZ       RELAY_LOOP
```

```
RELAY_HAVE_CMD:
```

```
            CJNE     A, #'5', RELAY_NOT_BACK
```

```

        CLR     REL1

        CLR     REL2

        LJMP    MENU

RELAY_NOT_BACK:

        CJNE    A, #'1', RELAY_NOT_R1ON

        SETB    REL1

        LCALL   RELAY_SHOW

        LJMP    RELAY_LOOP

RELAY_NOT_R1ON:

        CJNE    A, #'0', RELAY_NOT_R1OFF

        CLR     REL1

        LCALL   RELAY_SHOW

        LJMP    RELAY_LOOP

RELAY_NOT_R1OFF:

        CJNE    A, #'3', RELAY_NOT_R2ON

        SETB    REL2

        LCALL   RELAY_SHOW

        LJMP    RELAY_LOOP

RELAY_NOT_R2ON:

        CJNE    A, #'2', RELAY_LOOP

        CLR     REL2

        LCALL   RELAY_SHOW

        LJMP    RELAY_LOOP

RELAY_SHOW:

        PUSH    ACC

        MOV     A, #0C0H

        LCALL   COMNWRT

```

```

MOV      A, #'R'

LCALL    DATAWRT

MOV      A, #'1'

LCALL    DATAWRT

MOV      A, #':'

LCALL    DATAWRT

MOV      A, P3

ANL      A, #00000100B

JZ       RELAY_R1_ZERO

MOV      A, #'1'

SJMP     RELAY_R1_DONE

RELAY_R1_ZERO:

MOV      A, #'0'

RELAY_R1_DONE:

LCALL    DATAWRT

MOV      A, #' '

LCALL    DATAWRT

MOV      A, #'R'

LCALL    DATAWRT

MOV      A, #'2'

LCALL    DATAWRT

MOV      A, #':'

LCALL    DATAWRT

MOV      A, P3

ANL      A, #00001000B

JZ       RELAY_R2_ZERO

MOV      A, #'1'

SJMP     RELAY_R2_DONE

```

RELAY_R2_ZERO:

MOV A, #'0'

RELAY_R2_DONE:

LCALL DATAWRT

POP ACC

RET

; =====

; MODE 2: MORSE + 8 LED CONTROL

; Bluetooth commands:

; 1..8 = select Morse LED

; . = dot

; - = dash

; space = end letter and decode

; / = word space

; 0 = back to menu

; =====

MORSE_MODE:

LCALL RXBUF_INIT

MOV P1, #00H

MOV pattern, #00H

MOV length, #00H

MOV m_led_mask, #00H

MOV m_led_num, #00H

MOV line2_pos, #0C0H

LCALL LCD_CLR

MOV A, #80H

LCALL COMNWRT

```

MOV     DPTR, #MSG_MORSE1

LCALL   LCD_MSG

MOV     A, #0C0H

LCALL   COMNWRT

MOV     DPTR, #MSG_MORSE2

LCALL   LCD_MSG

MOV     DPTR, #MSG_BT_MORSE_READY

LCALL   UART_SEND_STR


MORSE_LOOP:

LCALL   REC_FUNC

CJNE    A, #'0', MORSE_CHK_DIGIT

LJMP    MENU


MORSE_CHK_DIGIT:

CJNE    A, #'1', MORSE_CHK2

MOV     m_led_mask, #01H

MOV     m_led_num, #'1'

LCALL   MORSE_SHOW_SEL

LJMP    MORSE_LOOP


MORSE_CHK2: CJNE    A, #'2', MORSE_CHK3

MOV     m_led_mask, #02H

MOV     m_led_num, #'2'

LCALL   MORSE_SHOW_SEL

LJMP    MORSE_LOOP


MORSE_CHK3: CJNE    A, #'3', MORSE_CHK4

MOV     m_led_mask, #04H

MOV     m_led_num, #'3'

```



```

        LCALL    MORSE_SHOW_SEL

        LJMP     MORSE_LOOP

MORSE_CHK4: CJNE    A, #'4', MORSE_CHK5

        MOV      m_led_mask, #08H

        MOV      m_led_num,  #'4'

        LCALL    MORSE_SHOW_SEL

        LJMP     MORSE_LOOP

MORSE_CHK5: CJNE    A, #'5', MORSE_CHK6

        MOV      m_led_mask, #10H

        MOV      m_led_num,  #'5'

        LCALL    MORSE_SHOW_SEL

        LJMP     MORSE_LOOP

MORSE_CHK6: CJNE    A, #'6', MORSE_CHK7

        MOV      m_led_mask, #20H

        MOV      m_led_num,  #'6'

        LCALL    MORSE_SHOW_SEL

        LJMP     MORSE_LOOP

MORSE_CHK7: CJNE    A, #'7', MORSE_CHK8

        MOV      m_led_mask, #40H

        MOV      m_led_num,  #'7'

        LCALL    MORSE_SHOW_SEL

        LJMP     MORSE_LOOP

MORSE_CHK8: CJNE    A, #'8', MORSE_CHK_DOT

        MOV      m_led_mask, #80H

        MOV      m_led_num,  #'8'

        LCALL    MORSE_SHOW_SEL

        LJMP     MORSE_LOOP

```

MORSE_CHK_DOT:

CJNE A, #'.', MORSE_CHK_DASH

LCALL MORSE_DOT

LJMP MORSE_LOOP

MORSE_CHK_DASH:

CJNE A, #'-', MORSE_CHK_SPACE

LCALL MORSE_DASH

LJMP MORSE_LOOP

MORSE_CHK_SPACE:

CJNE A, #' ', MORSE_CHK_SLASH

LCALL DECODE_LETTER

LJMP MORSE_LOOP

MORSE_CHK_SLASH:

CJNE A, #'/', MORSE_NOT_SLASH

LCALL DECODE_LETTER

MOV A, #' '

LCALL LINE2_APPEND

LJMP MORSE_LOOP

MORSE_NOT_SLASH:

LJMP MORSE_LOOP

MORSE_DOT:

MOV A, m_led_mask

JZ MD_RET

MOV P1, A

LCALL DELAY500MS

MOV P1, #00H

LCALL MORSE_SHOW_DOT

```

        MOV     A, pattern

        RL      A

        ANL     A, #1FH

        MOV     pattern, A

        INC     length

MD_RET:  RET

```

```

MORSE_DASH:

        MOV     A, m_led_mask

        JZ      MH_RET

        MOV     P1, A

        LCALL   DELAY1S

        MOV     P1, #00H

        LCALL   MORSE_SHOW_DASH

        MOV     A, pattern

        RL      A

        ANL     A, #1FH

        ORL     A, #01H

        MOV     pattern, A

        INC     length

MH_RET:  RET

```

```

MORSE_SHOW_SEL:

        MOV     A, #80H

        LCALL   COMNWRT

        MOV     DPTR, #MSG_SEL

        LCALL   LCD_MSG

        MOV     A, m_led_num

```

```
LCALL    DATAWRT

MOV      DPTR, #MSG_PAD6

LCALL    LCD_MSG

RET
```

MORSE_SHOW_DOT:

```
MOV      A, #80H

LCALL    COMNWRT

MOV      DPTR, #MSG_SYM_DOT

LCALL    LCD_MSG

RET
```

MORSE_SHOW_DASH:

```
MOV      A, #80H

LCALL    COMNWRT

MOV      DPTR, #MSG_SYM_DASH

LCALL    LCD_MSG

RET
```

DECODE_LETTER:

```
MOV      A, length

JZ       DL_EXIT

CLR      C

SUBB     A, #06H

JNC      DL_CLEAR

MOV      A, length

MOV      DPTR, #lengthOffset

MOVC     A, @A+DPTR
```

```

        MOV     tmp1, A

        MOV     A, pattern

        ADD     A, tmp1

        MOV     DPTR, #decodeTable

        MOVC    A, @A+DPTR

        JZ      DL_CLEAR

        LCALL   LINE2_APPEND

```

DL_CLEAR:

```

        MOV     pattern, #00H

        MOV     length, #00H

DL_EXIT:  RET

```

LINE2_APPEND:

```

        MOV     tmp2, A

        MOV     A, line2_pos

        LCALL   COMNWRT

        MOV     A, tmp2

        LCALL   DATAWRT

        INC     line2_pos

        MOV     A, line2_pos

        CJNE    A, #0D0H, L2_RET

        MOV     line2_pos, #0C0H

L2_RET:  RET

```

```

; =====

; MODE 3: ENCRYPT / DECRYPT

; Keypad digit = Caesar key (0..9)

; Bluetooth sends encrypted text

```

```
; '/' or CR or LF ends current message

; Spaces are preserved and output is shown on LCD line 2

; Keypad 5 returns to menu, any other key restarts mode

; =====
```

ENCRYPT_MODE:

```
    LCALL    RXBUF_INIT

    LCALL    LCD_CLR

    MOV      A, #80H

    LCALL    COMNWRT

    MOV      DPTR, #MSG_ENC1

    LCALL    LCD_MSG

    MOV      A, #0C0H

    LCALL    COMNWRT

    MOV      DPTR, #MSG_KEYIN

    LCALL    LCD_MSG

    ACALL    GET_KEY

    MOV      P2, #0FFH

    MOV      enc_key, A

    MOV      A, enc_key

    LCALL    DATAWRT

    MOV      A, enc_key

    CLR      C

    SUBB     A, #'0'

    MOV      enc_key, A

    LCALL    DELAY1S
```

ENC_START_MSG:

```
    LCALL    RXBUF_INIT
```

```

        LCALL    LCD_CLR

        MOV      A, #80H

        LCALL    COMNWRT

        MOV      DPTR, #MSG_ENC2

        LCALL    LCD_MSG

        MOV      line2_pos, #0C0H


ENC_LOOP:

        LCALL    REC_FUNC

        CJNE     A, #'/', ENC_CHK_CR

        SJMP     ENC_DONE


ENC_CHK_CR:

        CJNE     A, #0DH, ENC_CHK_LF

        SJMP     ENC_DONE


ENC_CHK_LF:

        CJNE     A, #0AH, ENC_CHK_SPACE

        SJMP     ENC_DONE


ENC_CHK_SPACE:

        CJNE     A, #' ', ENC_CHK_UPPER

        LCALL    LINE2_APPEND

        LJMP     ENC_LOOP


ENC_CHK_UPPER:

        MOV      tmp1, A

        CLR      C

        SUBB     A, #'A'

        JC       ENC_CHK_LOWER

        MOV      A, tmp1

```

```

        CLR        C

        SUBB       A, #('Z'+1)

        JNC        ENC_CHK_LOWER

        MOV        A, tmp1

        CLR        C

        SUBB       A, #'A'

        CLR        C

        SUBB       A, enc_key

        JNC        ENC_UPPER_OK

        ADD        A, #26

ENC_UPPER_OK:

        ADD        A, #'A'

        LCALL      LINE2_APPEND

        LJMP       ENC_LOOP


ENC_CHK_LOWER:

        MOV        A, tmp1

        CLR        C

        SUBB       A, #'a'

        JC         ENC_RAW

        MOV        A, tmp1

        CLR        C

        SUBB       A, #('z'+1)

        JNC        ENC_RAW

        MOV        A, tmp1

        CLR        C

        SUBB       A, #'a'

        CLR        C

```



```

        SUBB    A, enc_key

        JNC     ENC_LOWER_OK

        ADD     A, #26

ENC_LOWER_OK:

        ADD     A, #'A'

        LCALL   LINE2_APPEND

        LJMP    ENC_LOOP


ENC_RAW:

        MOV     A, tmp1

        LCALL   LINE2_APPEND

        LJMP    ENC_LOOP


ENC_DONE:

        MOV     A, #80H

        LCALL   COMNWRT

        MOV     DPTR, #MSG_ENC3

        LCALL   LCD_MSG

        ACALL   GET_KEY

        MOV     P2, #0FFH

        CJNE    A, #'5', ENC_RESTART

        LJMP    MENU


ENC_RESTART:

        LJMP    ENCRYPT_MODE


; =====

; MODE 4: ADVANCED 8-LED + BRIGHTNESS CONTROL

; Bluetooth / keypad commands:

```

```

; 1..8 = toggle LED1..LED8

; 9    = all LEDs ON

; 0    = all LEDs OFF

; .    = brightness up

; -    = brightness down

; /    = back to menu

; Brightness is global for the current LED mask.

; =====

```

```
LEDCTRL_MODE:
```

```

    LCALL  RXBUF_INIT

    MOV    P1, #00H

    MOV    led4_mask, #00H

    MOV    led4_bright, #09H

    MOV    led4_pwm, #00H

    LCALL  LCD_CLR

    MOV    A, #80H

    LCALL  COMNWRT

    MOV    DPTR, #MSG_LED41

    LCALL  LCD_MSG

    LCALL  LED4_SHOW

    MOV    DPTR, #MSG_BT_LED4_READY

    LCALL  UART_SEND_STR

```

```
LED4_LOOP:
```

```

    LCALL  LED4_PWM_SLICE

    LCALL  KEY_CHECK

    JNZ    LED4_HAVE_CMD

    LCALL  BT_CHECK

```

JZ LED4_LOOP

LED4_HAVE_CMD:

CJNE A, #'/', LED4_NOT_BACK

MOV P1, #00H

LJMP MENU

LED4_NOT_BACK:

CJNE A, #'1', LED4_CHK2

MOV A, led4_mask

XRL A, #01H

MOV led4_mask, A

LCALL LED4_SHOW

LJMP LED4_LOOP

LED4_CHK2: CJNE A, #'2', LED4_CHK3

MOV A, led4_mask

XRL A, #02H

MOV led4_mask, A

LCALL LED4_SHOW

LJMP LED4_LOOP

LED4_CHK3: CJNE A, #'3', LED4_CHK4

MOV A, led4_mask

XRL A, #04H

MOV led4_mask, A

LCALL LED4_SHOW

LJMP LED4_LOOP

LED4_CHK4: CJNE A, #'4', LED4_CHK5

MOV A, led4_mask

```

        XRL      A, #08H

        MOV      led4_mask, A

        LCALL    LED4_SHOW

        LJMP     LED4_LOOP

LED4_CHK5: CJNE   A, #'5', LED4_CHK6

        MOV      A, led4_mask

        XRL      A, #10H

        MOV      led4_mask, A

        LCALL    LED4_SHOW

        LJMP     LED4_LOOP

LED4_CHK6: CJNE   A, #'6', LED4_CHK7

        MOV      A, led4_mask

        XRL      A, #20H

        MOV      led4_mask, A

        LCALL    LED4_SHOW

        LJMP     LED4_LOOP

LED4_CHK7: CJNE   A, #'7', LED4_CHK8

        MOV      A, led4_mask

        XRL      A, #40H

        MOV      led4_mask, A

        LCALL    LED4_SHOW

        LJMP     LED4_LOOP

LED4_CHK8: CJNE   A, #'8', LED4_CHK_ALLON

        MOV      A, led4_mask

        XRL      A, #80H

        MOV      led4_mask, A

        LCALL    LED4_SHOW

        LJMP     LED4_LOOP

```

LED4_CHK_ALLON:

```
CJNE    A, #'9', LED4_CHK_ALLOFF

MOV     led4_mask, #0FFH

LCALL   LED4_SHOW

LJMP    LED4_LOOP
```

LED4_CHK_ALLOFF:

```
CJNE    A, #'0', LED4_CHK_BUP

MOV     led4_mask, #00H

LCALL   LED4_SHOW

LJMP    LED4_LOOP
```

LED4_CHK_BUP:

```
CJNE    A, #'.', LED4_CHK_BDN

MOV     A, led4_bright

CJNE    A, #09H, LED4_BUP_DO

LJMP    LED4_LOOP
```

LED4_BUP_DO:

```
INC     led4_bright

LCALL   LED4_SHOW

LJMP    LED4_LOOP
```

LED4_CHK_BDN:

```
CJNE    A, #'-', LED4_IGNORE

MOV     A, led4_bright

JZ      LED4_BDN_EXIT

DEC     led4_bright

LCALL   LED4_SHOW
```

LED4_BDN_EXIT:

```
LJMP    LED4_LOOP
```

LED4_IGNORE:

LJMP LED4_LOOP

LED4_PWM_SLICE:

MOV A, led4_pwm

INC A

CJNE A, #0AH, LED4_PWM_SAVE

MOV A, #00H

LED4_PWM_SAVE:

MOV led4_pwm, A

CJNE A, led4_bright, LED4_PWM_NE

SJMP LED4_PWM_OFF

LED4_PWM_NE:

JC LED4_PWM_ON

LED4_PWM_OFF:

MOV P1, #00H

LCALL LED4_PWM_DELAY

RET

LED4_PWM_ON:

MOV A, led4_mask

MOV P1, A

LCALL LED4_PWM_DELAY

RET

LED4_PWM_DELAY:

MOV R4, #08

```

LED4_PD1:  MOV     R5, #80

LED4_PD2:  DJNZ    R5, LED4_PD2

           DJNZ    R4, LED4_PD1

           RET

LED4_SHOW:

           MOV     A, #0C0H

           LCALL   COMNWRT

           MOV     A, #'B'

           LCALL   DATAWRT

           MOV     A, #':'

           LCALL   DATAWRT

           MOV     A, led4_bright

           ADD     A, #'0'

           LCALL   DATAWRT

           MOV     A, #' '

           LCALL   DATAWRT

           MOV     A, #'M'

           LCALL   DATAWRT

           MOV     A, #':'

           LCALL   DATAWRT

           MOV     A, led4_mask

           SWAP    A

           LCALL   HEX_NIBBLE_ASC

           LCALL   DATAWRT

           MOV     A, led4_mask

           LCALL   HEX_NIBBLE_ASC

           LCALL   DATAWRT

```

```

MOV     A, #' '

LCALL   DATAWRT

MOV     A, # '/'

LCALL   DATAWRT

MOV     A, #'='

LCALL   DATAWRT

MOV     A, #'B'

LCALL   DATAWRT

MOV     A, #'k'

LCALL   DATAWRT

MOV     A, #' '

LCALL   DATAWRT

MOV     A, #' '

LCALL   DATAWRT

MOV     A, #' '

LCALL   DATAWRT

RET

```

HEX_NIBBLE_ASC:

```

ANL     A, #0FH

CLR     C

SUBB    A, #0AH

JC      HN_DEC

ADD     A, #'A'

RET

```

HN_DEC:

```

ADD     A, #('0'+10)

RET

```



```
; =====
```

```
; LCD ROUTINES
```

```
; =====
```

```
LCD_INIT:
```

```
    LCALL    LONGDELAY
```

```
    LCALL    LONGDELAY
```

```
    MOV      A, #38H
```

```
    LCALL    COMNWRT
```

```
    MOV      A, #38H
```

```
    LCALL    COMNWRT
```

```
    MOV      A, #38H
```

```
    LCALL    COMNWRT
```

```
    MOV      A, #0FH
```

```
    LCALL    COMNWRT
```

```
    MOV      A, #01H
```

```
    LCALL    COMNWRT
```

```
    LCALL    LONGDELAY
```

```
    MOV      A, #06H
```

```
    LCALL    COMNWRT
```

```
    RET
```

```
LCD_CLR:
```

```
    MOV      A, #01H
```

```
    LCALL    COMNWRT
```

```
    LCALL    LONGDELAY
```

```
    RET
```

LCD_MSG:

CLR A

LCD_NEXT:

MOVC A, @A+DPTR

JZ LCD_DONE

LCALL DATAWRT

INC DPTR

CLR A

SJMP LCD_NEXT

LCD_DONE: RET

COMNWRT:

MOV LCD_DATA, A

CLR RS

CLR RW

SETB E

LCALL DELAY

CLR E

LCALL DELAY

RET

DATAWRT:

MOV LCD_DATA, A

SETB RS

CLR RW

SETB E

LCALL DELAY

CLR E

```
LCALL    DELAY
```

```
RET
```

```
; =====
```

```
; UART
```

```
; =====
```

```
UART_INIT:
```

```
CLR      TR1
```

```
SETB     ES
```

```
MOV      TMOD, #20H
```

```
MOV      TH1,  #0FDH
```

```
MOV      TL1,  #0FDH
```

```
MOV      SCON, #50H
```

```
CLR      TI
```

```
CLR      RI
```

```
SETB     TR1
```

```
SETB     EA
```

```
RET
```

```
RXBUF_INIT:
```

```
MOV      rx_head, #50H
```

```
MOV      rx_tail, #50H
```

```
MOV      rx_count, #00H
```

```
RET
```

```
RXBUF_GET:
```

```
MOV      A, rx_count
```

```
JZ       RXG_EMPTY
```

```

        DEC     rx_count

        MOV     R0, rx_tail

        MOV     A, @R0

        INC     R0

        CJNE    R0, #60H, RXG_OK

        MOV     R0, #50H

RXG_OK:

        MOV     rx_tail, R0

        RET

RXG_EMPTY:

        MOV     A, #00H

        RET

; Blocking receive with measured normalization

REC_FUNC:

RF_WAIT:

        MOV     A, rx_count

        JZ      RF_WAIT

        LCALL   RXBUF_GET

        LCALL   NORMALIZE_RX

        JZ      RF_WAIT

        RET

; Non-blocking receive with measured normalization

BT_CHECK:

BT_NEXT:

        MOV     A, rx_count

        JZ      BT_NONE

```

```

        LCALL    RXBUF_GET

        LCALL    NORMALIZE_RX

        JZ       BT_NEXT

        RET

BT_NONE:

        MOV      A, #00H

        RET

; Measured bytes from hardware/app:

; A -> 81h, 1 -> 71h, 8 -> 78h

; . -> 4Eh, - -> 4Dh, space -> 40h, / -> 4Fh, 0 -> 70h

NORMALIZE_RX:

        CJNE     A, #0DH, NRX_CHKLF

        MOV      A, #00H

        RET

NRX_CHKLF:  CJNE     A, #0AH, NRX_CHKDOT

        MOV      A, #00H

        RET

NRX_CHKDOT: CJNE     A, #04EH, NRX_CHKDASH

        MOV      A, # '.'

        RET

NRX_CHKDASH:

        CJNE     A, #04DH, NRX_CHKSP

        MOV      A, # '-'

        RET

NRX_CHKSP:  CJNE     A, #040H, NRX_CHKSL

        MOV      A, # ' '

        RET

```

NRX_CHKSL: CJNE A, #04FH, NRX_SUB40

MOV A, #'/'

RET

NRX_SUB40:

CLR C

SUBB A, #040H

RET

UART_SEND:

CLR TI

MOV SBUF, A

US_WAIT: JNB TI, US_WAIT

CLR TI

RET

UART_SEND_STR:

CLR A

USS_NEXT: MOVC A, @A+DPTR

JZ USS_DONE

LCALL UART_SEND

INC DPTR

CLR A

SJMP USS_NEXT

USS_DONE: RET

; =====

; KEYPAD

; =====

GET_KEY:

GK0: MOV P2, #0FFH

GK1: CLR P2.4

 CLR P2.5

 CLR P2.6

 CLR P2.7

 MOV A, P2

 ANL A, #0FH

 CJNE A, #0FH, GK1

GK2: LCALL DELAY

 MOV A, P2

 ANL A, #0FH

 CJNE A, #0FH, GK_DEB

 SJMP GK2

GK_DEB: LCALL DELAY

 MOV A, P2

 ANL A, #0FH

 CJNE A, #0FH, GK_SCAN

 SJMP GK2

GK_SCAN:

 MOV P2, #0FFH

 CLR P2.4

 MOV A, P2

 ANL A, #0FH

 CJNE A, #0FH, GK_R0

 MOV P2, #0FFH

 CLR P2.5

 MOV A, P2

```

        ANL      A, #0FH

        CJNE     A, #0FH, GK_R1

        MOV      P2, #0FFH

        CLR      P2.6

        MOV      A, P2

        ANL      A, #0FH

        CJNE     A, #0FH, GK_R2

        MOV      P2, #0FFH

        CLR      P2.7

        MOV      A, P2

        ANL      A, #0FH

        CJNE     A, #0FH, GK_R3

        SJMP     GK2

GK_R0:   MOV      DPTR, #KCODE0

        SJMP     GK_FIND

GK_R1:   MOV      DPTR, #KCODE1

        SJMP     GK_FIND

GK_R2:   MOV      DPTR, #KCODE2

        SJMP     GK_FIND

GK_R3:   MOV      DPTR, #KCODE3

GK_FIND: RRC      A

        JNC      GK_MATCH

        INC      DPTR

        SJMP     GK_FIND

GK_MATCH: CLR      A

        MOVC     A, @A+DPTR

        MOV      P2, #0FFH

        RET

```


KEY_CHECK:

```
MOV    P2, #0FFH

CLR    P2.4

CLR    P2.5

CLR    P2.6

CLR    P2.7

MOV    A, P2

ANL    A, #0FH

CJNE   A, #0FH, KC_SCAN

MOV    P2, #0FFH

MOV    A, #00H

RET
```

KC_SCAN:

```
MOV    P2, #0FFH

CLR    P2.4

MOV    A, P2

ANL    A, #0FH

CJNE   A, #0FH, KC_R0

MOV    P2, #0FFH

CLR    P2.5

MOV    A, P2

ANL    A, #0FH

CJNE   A, #0FH, KC_R1

MOV    P2, #0FFH

CLR    P2.6

MOV    A, P2

ANL    A, #0FH
```

```

        CJNE    A, #0FH, KC_R2

        MOV     P2, #0FFH

        CLR     P2.7

        MOV     A, P2

        ANL     A, #0FH

        CJNE    A, #0FH, KC_R3

        MOV     P2, #0FFH

        MOV     A, #00H

        RET

KC_R0:    MOV     DPTR, #KCODE0

        SJMP    KC_FIND

KC_R1:    MOV     DPTR, #KCODE1

        SJMP    KC_FIND

KC_R2:    MOV     DPTR, #KCODE2

        SJMP    KC_FIND

KC_R3:    MOV     DPTR, #KCODE3

KC_FIND:  RRC     A

        JNC     KC_MATCH

        INC     DPTR

        SJMP    KC_FIND

KC_MATCH: CLR     A

        MOVC    A, @A+DPTR

        PUSH    ACC

KC_WAITREL: MOV    P2, #0FFH

        CLR     P2.4

        CLR     P2.5

        CLR     P2.6

        CLR     P2.7

```

```

MOV    tmp1, P2

MOV    A, tmp1

ANL    A, #0FH

CJNE   A, #0FH, KC_WAITREL

MOV    P2, #0FFH

POP    ACC

RET

```

```

; =====

```

```

; DELAYS

```

```

; =====

```

```

DELAY:

```

```

        MOV    R3, #50

D1:      MOV    R5, #255

D2:      DJNZ   R5, D2

        DJNZ   R3, D1

        RET

```

```

LONGDELAY:

```

```

        MOV    R3, #200

LD1:     MOV    R5, #255

LD2:     DJNZ   R5, LD2

        DJNZ   R3, LD1

        RET

```

```

DELAY1S:

```

```

        MOV    R3, #20

S1:      MOV    R2, #250

```

```

S2:      MOV     R5, #200

S3:      DJNZ    R5, S3

          DJNZ    R2, S2

          DJNZ    R3, S1

          RET

```

DELAY500MS:

```

          MOV     R3, #10

H1:      MOV     R2, #250

H2:      MOV     R5, #200

H3:      DJNZ    R5, H3

          DJNZ    R2, H2

          DJNZ    R3, H1

          RET

```

```

; =====

```

```

; TABLES / STRINGS

```

```

; =====

```

```

          ORG     1200H

```

lengthOffset:

```

          DB      0, 2, 4, 8, 16, 32

```

decodeTable:

```

          DB      0, 0

          DB      'E', 'T'

          DB      'I', 'A', 'N', 'M'

          DB      'S', 'U', 'R', 'W', 'D', 'K', 'G', 'O'

          DB      'H', 'V', 'F', 0, 'L', 0, 'P', 'J'

```

```

        DB      'B','X','C','Y','Z','Q',0,0
        DB      '5','4',0,'3',0,'2',0,'1'
        DB      '6',0,0,'7',0,'8','9','0'
        DB      0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
        DB      0,0,0,0,0,0,0,0,0,0,0,0,0,0,0

        ORG      1400H

MSG1:      DB      'EEE 4706 Project',0
MSG2:      DB      'Hardware Final ',0
MSG_PIN:    DB      'Enter PIN:      ',0
MSG_OK:     DB      'Access Granted! ',0
MSG_WELCOME:DB      'Welcome          ',0
MSG_WRONG:  DB      'Wrong PIN!       ',0
MSG_LEFT:   DB      'Tries left:      ',0
MSG_LOCK:   DB      '*** LOCKED **    ',0
MSG_WAIT:   DB      'Wait 10 seconds ',0
MSG_MENU1:  DB      'Select a mode   ',0
MSG_MENU2:  DB      '1R 2M 3E 4LED ',0
MSG_RELAY1: DB      'Relay mode     ',0
MSG_MORSE1: DB      'Morse mode     ',0
MSG_MORSE2: DB      'Send 1..8 / .- ',0
MSG_ENC1:   DB      'Decrypt key?   ',0
MSG_ENC2:   DB      'Send txt end / ',0
MSG_ENC3:   DB      'Done 5=back    ',0
MSG_KEYIN:  DB      'Enter key 0..9 ',0
MSG_BACK5:  DB      '5=back else rpt ',0
MSG_SEL:    DB      'SEL LED:',0
MSG_PAD6:   DB      '          ',0

```

```

MSG_SYM_DOT:DB      'SYM:.'      ',0

MSG_SYM_DASH:DB     'SYM:-'      ',0

MSG_BT_REL_READY:

        DB          'Relay ready',0DH,0AH,0

MSG_BT_MORSE_READY:

        DB          'Morse ready',0DH,0AH,'1..8 select LED',0DH,0AH,'. - sp / 0',0DH,0AH,0

MSG_BT_LED4_READY:

        DB          'LED ctrl ready',0DH,0AH

        DB          '1..8 toggle 9=all on',0DH,0AH

        DB          '0=all off .=B+ -=B-',0DH,0AH

        DB          '/=back',0DH,0AH,0


        ORG         1800H

KCODE0:   DB        '1','2','3','-'

KCODE1:   DB        '4','5','6','*'

KCODE2:   DB        '7','8','9','/'

KCODE3:   DB        'C','0','=','+'

        END

```

8. Hardware Implementation

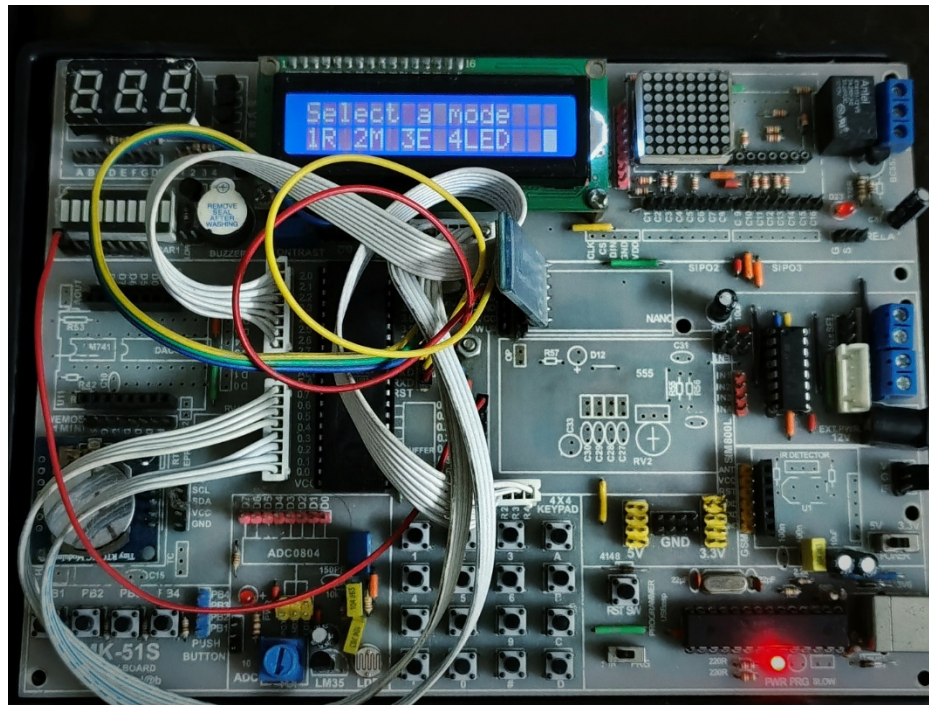


Fig: Hardware in running condition. This is the part where we select the mode of operation

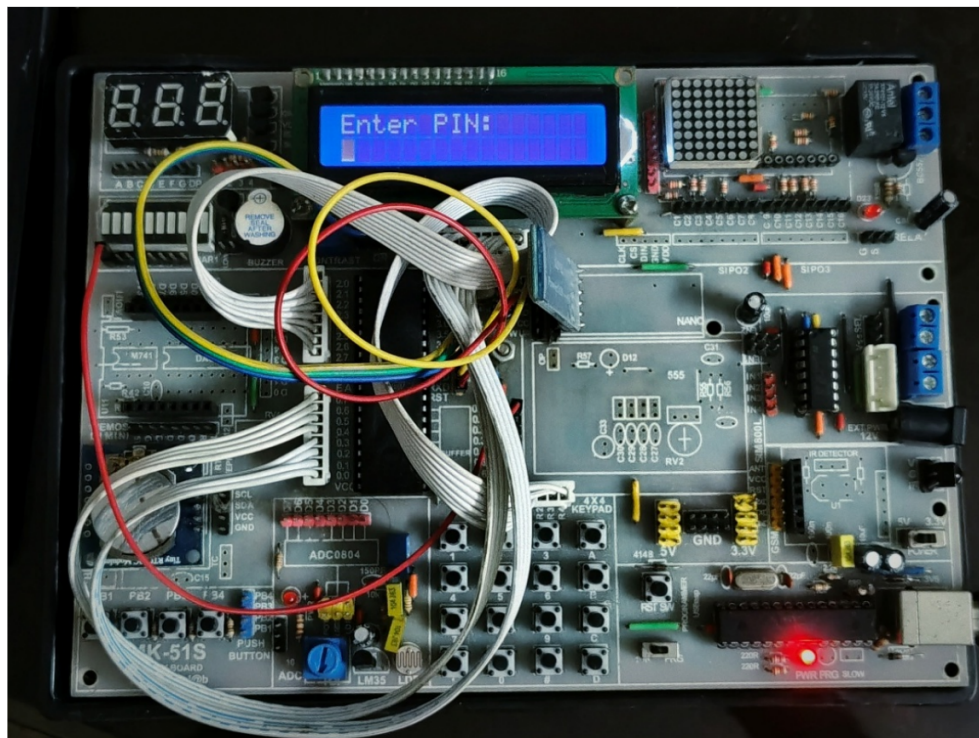
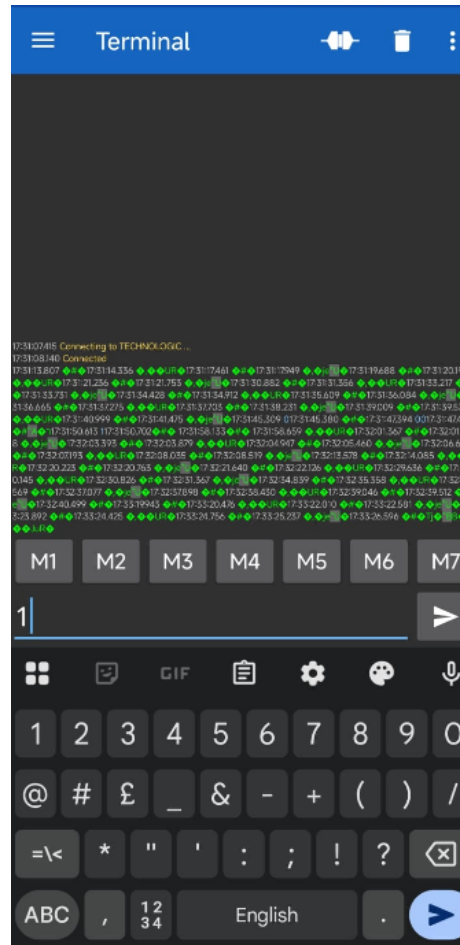


Fig: Password protection

9. Problems Faced

While transmitting data from the microcontroller to the mobile application through Bluetooth, the received output on the mobile app appeared as garbage or unreadable characters instead of the expected data.



This issue occurred due to improper handling of serial communication and mismatch in data formatting between the microcontroller and the mobile application. The problem was solved by carefully debugging the UART communication section of the code, correcting the data handling process, and ensuring proper formatting and normalization of transmitted characters. After making these corrections, the data was displayed correctly on the mobile application.

During the development of the project, all individual features such as relay control, Morse code transmission, encryption and decryption, LED brightness control, and password protection were working correctly when tested separately. However, when combining all these features into a single system, it became difficult to integrate them together and resulted in unexpected behavior. This was mainly due to conflicts between different sections of the code, shared use of ports and variables, and improper program flow between modes. This problem was solved by restructuring the code into proper modular sections, carefully managing memory and port usage, and testing

each module step by step after integration. By doing this, all features were successfully integrated into one complete system.

10. Conclusion

This project successfully demonstrates a multi-mode wireless control system using the 8051 microcontroller. All required features—LED control, Morse transmission, relay switching, decryption, password protection, and PWM brightness—were implemented and tested.

The interrupt-driven communication ensured reliable Bluetooth data handling, while efficient use of memory and registers enabled implementation of features like Morse decoding and Caesar cipher processing within hardware constraints.

The project strengthened practical skills in embedded system design, Assembly programming, and hardware interfacing. It also provides a strong foundation for future improvements such as enhanced encryption, additional sensors, or extended wireless capabilities.